pfi

# Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

# Case Study: Consumer Credit

WEEK 4 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# The problem and the data

Real public data: the UCI Default of Credit Card Clients dataset

**30,000**
credit card accounts

**25**
columns in the file

**6**
months of records

The first three accounts, seven of the 25 columns. Records from April to September 2005.

| ID | LIMIT_BAL | SEX | EDUCATION | AGE | PAY_0 | default payment next month |
|---|---|---|---|---|---|---|
| 1 | 20000 | 2 | 2 | 24 | 2 | 1 |
| 2 | 120000 | 2 | 2 | 26 | -1 | 1 |
| 3 | 90000 | 2 | 2 | 34 | 0 | 0 |

```
raw = pd.read_csv(data_folder + 'credit_card_default.csv')
print(raw.shape)   # (30000, 25)
```

**Amounts are in New Taiwan dollars (NT dollars). PAY_0 is September 2005, back to PAY_6, April. There is no PAY_1.**

The colleague asks

**What does this file show about which accounts defaulted the following month?**

**The answer is a description: rates in this file, each with its count. It does not say why an account defaulted, and it is not advice on lending.**

### Taiwan, 2005

The file is one set of credit card accounts, in Taiwan, in 2005. Consumer credit markets differ from one country and one period to another, including from US credit. Nothing found here carries over to another market.

Source: Yeh, I. (2009). Default of Credit Card Clients [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C55S3H.
License: Creative Commons Attribution 4.0 International (CC BY 4.0), https://creativecommons.org/licenses/by/4.0/legalcode
Changes made for this course: The original is an Excel file with two header rows. The first, which holds the labels X1 to X23 and Y, was dropped, and the sheet was saved as credit_card_default.csv. Nothing else was changed: all 30,000 rows and 25 columns are there with their original names and values.

# The file against its documentation

| Column | The documentation lists | The file also holds | Accounts holding them | Decision |
|---|---|---|---|---|
| SEX | 1, 2 | nothing else | 0 | Nothing to decide |
| EDUCATION | 1, 2, 3, 4 | 0, 5, 6 | 345 | Kept, labelled not documented |
| MARRIAGE | 1, 2, 3 | 0 | 54 | Kept, labelled not documented |
| PAY_0, September status | -1, and 1 to 9 | -2, 0 | 17,496 | Kept as code -2 and code 0 |
| default payment next month | 1, 0 | nothing else | 0 | Renamed default in the working copy |

**A code the documentation does not define is kept, counted, and called not documented. It is not dropped or guessed at.**

```
education_labels = {1: 'graduate school', 2: 'university',
                    3: 'high school', 4: 'others'}
cards['education'] = cards['EDUCATION'].map(education_labels)
print(cards['education'].isna().sum())   # 345
cards['education'] = cards['education'].fillna('not documented')
```

### A name with a space cannot follow the dot

raw.default payment next month.mean() stops with SyntaxError: invalid syntax.
raw['default payment next month'] works.

**map() left 345 blanks and raised no error. isna().sum() caught it.**
Nothing is missing. 35 rows repeat once the ID is left out: kept.

### The same in Excel: sheet cards, rows 2 to 30001, EDUCATION in column D

`=COUNTIF(D2:D30001, 5)` returns 280 `=COUNTIF(D2:D30001, "<1") + COUNTIF(D2:D30001, ">4")` returns 345

`=IFERROR(VLOOKUP(D2, $AB$2:$AC$5, 2, FALSE), "not documented")` the label. Codes in AB2 to AB5, meanings in AC2 to AC5.

Every code at once: a pivot table with EDUCATION in Rows and Count of ID in Values. It has seven rows.

# The outcome: a rate and its count

The outcome holds 1 for a default and 0 for none

**6,636**
defaults: the sum

/

**30,000**
rows: the count

=

**0.2212**
the mean: a rate

```
print(cards['default'].mean())       # 0.2212
defaults = cards['default'].sum()    # 6636
accounts = len(cards)                # 30000
```

**The mean of a 0/1 column is the share of rows that hold 1.**

22.12 percent of the accounts in this file defaulted the following month.

**The same in Excel, with the outcome in column Y**

| | |
|---|---|
| `=SUM(Y2:Y30001)` | returns 6,636, the defaults |
| `=COUNT(Y2:Y30001)` | returns 30,000, the accounts |
| `=AVERAGE(Y2:Y30001)` | returns 0.2212. As a percentage: 22.12% |

A rate needs its count

| Group | Defaults | Accounts | Rate, percent |
|---|---|---|---|
| EDUCATION code 0 | 0 | 14 | 0.00 |
| All accounts | 6,636 | 30,000 | 22.12 |

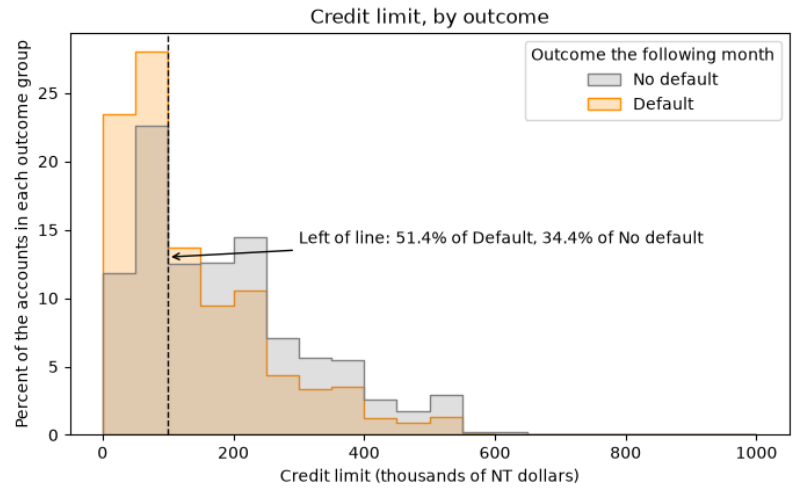
0 defaults among 14 accounts. 1 default in 14 would be 7.14 percent. On 30,000 accounts, one account more or less moves the rate by 0.003 of a percentage point.

```
code_zero = cards[cards['EDUCATION'] == 0]
print(code_zero['default'].mean() * 100)   # 0.0
print(len(code_zero))                      # 14
```

**From here on, no rate is shown without the number of accounts behind it.**

# One column at a time

Credit limit, one shape for each outcome




51.4 percent of the 6,636 accounts that defaulted had a limit below 100,000 NT dollars, and 34.4 percent of the 23,364 that did not. Shares of a group, not default rates.

### In Excel: limit in B, bill in M, payment in S

`=AVERAGE(B2:B30001)` returns 167,484
`=MEDIAN(B2:B30001)` returns 140,000
`=SKEW(S2:S30001)` returns 14.67

Chart: Insert, Insert Statistic Chart, Histogram.

Four columns: amounts in NT dollars, age in years

| Column | What it is | Median | Mean | Skew | Lowest | Highest |
|---|---|---|---|---|---|---|
| LIMIT_BAL | credit limit | 140,000 | 167,484 | 0.99 | 10,000 | 1,000,000 |
| AGE | age | 34 | 35.5 | 0.73 | 21 | 79 |
| BILL_AMT1 | September bill | 22,382 | 51,223 | 2.66 | -165,580 | 964,511 |
| PAY_AMT1 | paid in September | 2,100 | 5,664 | 14.67 | 0 | 873,552 |

81 different limits. Most common: 50,000, on 3,365 accounts. 206 are above 500,000.

### The September bill: what the documentation does not explain

590 bills are below zero, 2,008 are exactly zero, 2,115 are above the limit.

**Kept, counted, and reported as they are. No guess is made.**

5,249 accounts paid nothing in September, and 471 paid more than 50,000. With a skew of 14.67, quote the median and the quartiles.

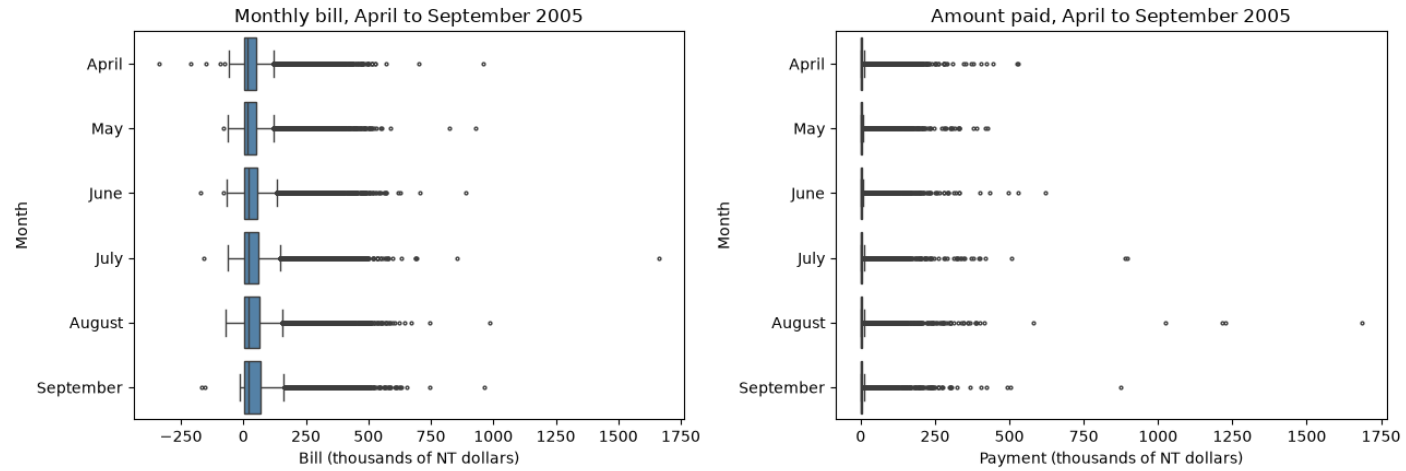
```
sns.histplot(data=cards, x='limit_thousands', hue='outcome',
             stat='percent', common_norm=False, element='step')
print((cards['BILL_AMT1'] > cards['LIMIT_BAL']).sum())   # 2115
```

`=COUNTIF(M2:M30001, "<0")` returns 590 `=COUNTIF(S2:S30001, 0)` returns 5,249

`=SUMPRODUCT(--(M2:M30001 > B2:B30001))` returns 2,115, the bills above the limit

# Far from the rest

Monthly bill and amount paid, April to September 2005, in thousands of NT dollars




## How far is far

The fences sit 1.5 times the interquartile range (IQR) below the first quartile and above the third. The box plot draws them as its whiskers.

The largest July bill is 1,664 thousand NT dollars and the largest August payment 1,684 thousand. One account each.

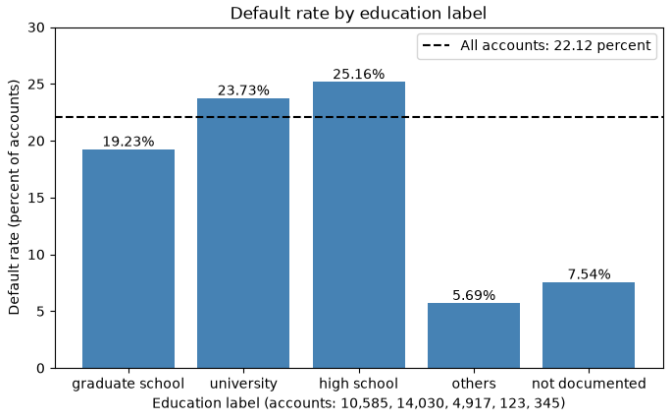
Accounts beyond the fences, in NT dollars. Default rates in percent.

| Column | Lower fence | Upper fence | Accounts above | Default rate above | Default rate, the rest |
|---|---|---|---|---|---|
| LIMIT_BAL | -235,000 | 525,000 | 167 | 11.38 | 22.18 |
| BILL_AMT1 | -91,740 | 162,389 | 2,398 | 19.89 | 22.31 |
| PAY_AMT1 | -5,009 | 11,015 | 2,745 | 12.28 | 23.11 |

**Far from the middle is a statement about distance. It is not a statement about whether the row matters. Nothing is dropped.**

# The outcome against one column

Default rate by education label, with a reference line




**Counts are not rates: the group with more defaults had the lower rate.**

A bar on 123 accounts looks as solid as one on 14,030: read the counts.

```
by_sex = cards.groupby('sex')['default'].agg(['count', 'sum', 'mean'])
by_sex.columns = ['accounts', 'defaults', 'rate_pct']
```

**In Excel: SEX in column C, the outcome in column Y**

| | |
|---|---|
| `=COUNTIF(C2:C30001, 1)` | returns 11,888, the accounts |
| `=SUMIF(C2:C30001, 1, Y2:Y30001)` | returns 2,873, the defaults |
| `=AVERAGEIF(C2:C30001, 1, Y2:Y30001)` | returns 0.2417, the rate |

| Column | Label | accounts | defaults | rate_pct |
|---|---|---|---|---|
| sex | female | 18,112 | 3,763 | 20.78 |
| sex | male | 11,888 | 2,873 | 24.17 |
| education | graduate school | 10,585 | 2,036 | 19.23 |
| education | university | 14,030 | 3,330 | 23.73 |
| education | high school | 4,917 | 1,237 | 25.16 |
| education | others | 123 | 7 | 5.69 |
| education | not documented | 345 | 26 | 7.54 |
| marriage | married | 13,659 | 3,206 | 23.47 |
| marriage | single | 15,964 | 3,341 | 20.93 |
| marriage | others | 323 | 84 | 26.01 |
| marriage | not documented | 54 | 5 | 9.26 |

**Differences between groups in this file do not explain themselves, and they are not a basis for treating people differently.**

In each grouping the accounts add up to 30,000.

# A rough margin of error

From a count to percentage points

```
margin_pts = 1.96 x sqrt(p x (1 - p) / n) x 100
```

p is the rate as a fraction, n the number of accounts.

```
p = table['rate_pct']      # still a fraction here
margin = 1.96 * np.sqrt(p * (1 - p) / table['accounts'])
table['margin_pts'] = (margin * 100).round(2)
```

**Under the square root: four times the accounts halves the margin.**

With 0 defaults in 14 accounts the formula returns 0.00. That is the formula failing, not a certain rate.

### In Excel: the rate in AB2, the accounts in AA2

`=1.96*SQRT(AB2*(1-AB2)/AA2)` returns 0.0077: 0.77 points (male row)

| Group | accounts | rate_pct | margin_pts |
|---|---|---|---|
| all accounts | 30,000 | 22.12 | 0.47 |
| sex: male | 11,888 | 24.17 | 0.77 |
| sex: female | 18,112 | 20.78 | 0.59 |
| education: university | 14,030 | 23.73 | 0.70 |
| education: high school | 4,917 | 25.16 | 1.21 |
| education: others | 123 | 5.69 | 4.09 |
| marriage: others | 323 | 26.01 | 4.78 |
| marriage: not documented | 54 | 9.26 | 7.73 |

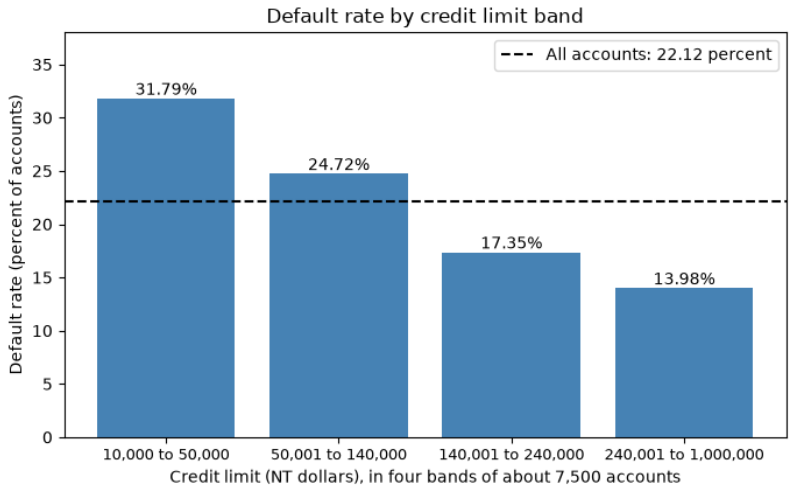
Rates in percent. Margins in percentage points, give or take.

**A rough guide. It treats the accounts as a random draw from a larger set of accounts like them, and the dataset page does not say how these accounts were chosen.**

It is about chance in the count and nothing else. It does not say why a rate is what it is.

# Bands: pd.cut() and pd.qcut()

Default rate by credit limit band, with the reference line




Each band has more than 7,000 accounts: no rate rests on a thin group.

An association. The documentation does not say how the limits were set.

pd.qcut(cards['PAY_AMT1'], 10): ValueError, Bin edges must be unique. 5,249 accounts paid nothing, so two edges are both 0.

| limit_band | accounts | defaults | rate_pct |
|---|---|---|---|
| 10,000 to 50,000 | 7,676 | 2,440 | 31.79 |
| 50,001 to 140,000 | 7,614 | 1,882 | 24.72 |
| 140,001 to 240,000 | 7,643 | 1,326 | 17.35 |
| 240,001 to 1,000,000 | 7,067 | 988 | 13.98 |

**pd.qcut(): the data chooses the edges, at the quartiles**

```
limit_band = pd.qcut(cards['LIMIT_BAL'], 4)
```

7,676, 7,614, 7,643, and 7,067 accounts, not 7,500 each: the 3,365 accounts at exactly 50,000 stay in one band.

**pd.cut(): you choose the edges**

```
pd.cut(cards['AGE'], bins=[20, 30, 40, 50, 60, 80], labels=age_names)
```

Five age bands: 11,013, 10,713, 6,005, 1,997, and 272 accounts.

`=PERCENTILE.INC(B2:B30001, 0.25)` returns 50,000

`=PERCENTILE.INC(S2:S30001, 0.1)` returns 0, the same as the minimum payment

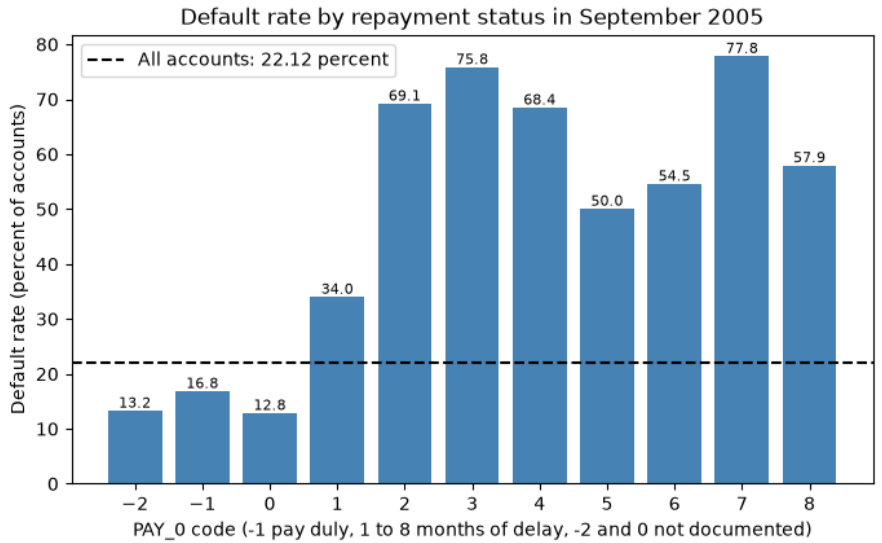
`=AVERAGEIFS(Y2:Y30001, B2:B30001, ">50000", B2:B30001, "<=140000")` returns 0.2472

`=VLOOKUP(B2, $AH$2:$AI$5, 2, TRUE)` returns the band

`=COUNTIFS(B2:B30001, ">50000", B2:B30001, "<=140000")` returns 7,614

Lowest limit of each band in AH2 to AH5, band names in AI2 to AI5.

# Repayment status in September




```
by_status = rate_table('PAY_0')
```

Codes -2, -1, and 0 are below the line. Every code from 1 up is above it.

Codes -2 and 0: the two lowest rates, and 17,496 accounts between them.

**What they mean is not in the documentation, and the rates do not say.**

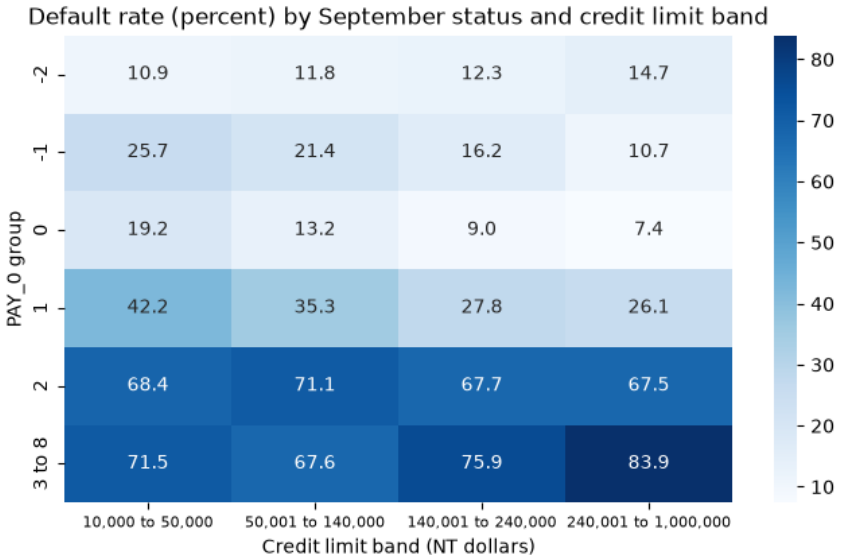
| PAY_0 | The documentation | accounts | defaults | rate_pct |
|---|---|---|---|---|
| -2 | does not define it | 2,759 | 365 | 13.23 |
| -1 | pay duly | 5,686 | 954 | 16.78 |
| 0 | does not define it | 14,737 | 1,888 | 12.81 |
| 1 | delay of one month | 3,688 | 1,252 | 33.95 |
| 2 | delay of two months | 2,667 | 1,844 | 69.14 |
| 3 | delay of three months | 322 | 244 | 75.78 |
| 4 | delay of four months | 76 | 52 | 68.42 |
| 5 | delay of five months | 26 | 13 | 50.00 |
| 6 | delay of six months | 11 | 6 | 54.55 |
| 7 | delay of seven months | 9 | 7 | 77.78 |
| 8 | delay of eight months | 19 | 11 | 57.89 |

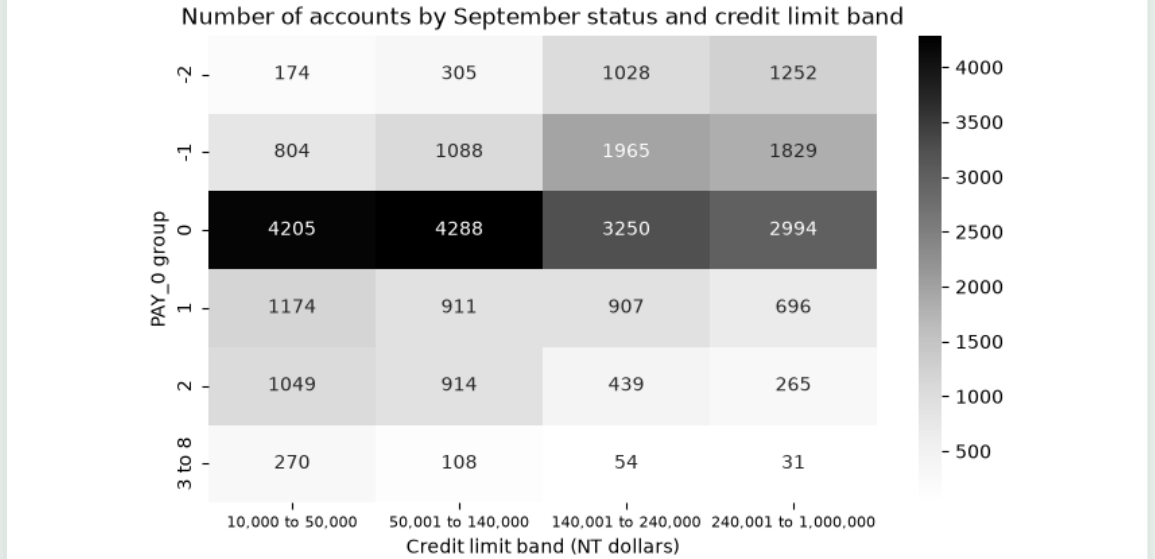
**Codes 4 to 8 hold 141 accounts between them: thin groups.**

Code 6: 54.55 percent on 11 accounts, give or take about 29 points.

`=COUNTIF(G2:G30001, -2) + COUNTIF(G2:G30001, 0)` returns 17,496: the accounts with code -2 or code 0 in September

# Two columns: the rate and the count




```
rate_pivot = pd.pivot_table(cards, values='default', index='status_group',
                            columns='limit_band', aggfunc='mean')
```

**The highest rate, 83.9 percent, is on the smallest cell: 31 accounts, give or take about 12.9 points.**

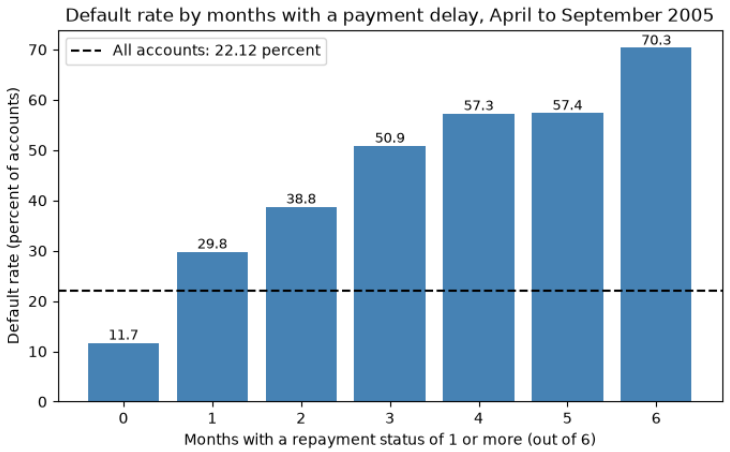
Status 2, and 3 to 8: 67.5 to 83.9 percent in every limit band. Code 0: 19.2 to 7.4 percent. 24 cells, 31 to 4,288 accounts, 30,000 in all.

`=COUNTIFS(G2:G30001, 2, B2:B30001, "<=50000")` returns 1,049 `=AVERAGEIFS(Y2:Y30001, G2:G30001, 2, B2:B30001, "<=50000")` returns 0.684

Pivot table: status group in Rows, limit band in Columns, outcome in Values by Average or Count. Colours: Conditional Formatting, Color Scales.

# Derived columns

Months with a payment delay: a status code of 1 or more




**No month of delay: 11.71 percent of 19,931 accounts defaulted.**

**All six months: 70.32 percent of 1,341 accounts defaulted.**

By outcome: 35.2 percent of the 6,636 accounts that defaulted had no month of delay. A percentage is a share of some total. Say which.

```
cards['months_delayed'] = (cards[pay_columns] >= 1).sum(axis=1)
```

Utilisation: September bill / credit limit. Median 0.314.

| utilisation_band | accounts | defaults | rate_pct |
|---|---|---|---|
| zero or below | 2,598 | 643 | 24.75 |
| above 0 to 0.25 | 11,440 | 1,923 | 16.81 |
| above 0.25 to 0.5 | 3,588 | 780 | 21.74 |
| above 0.5 to 0.75 | 3,579 | 929 | 25.96 |
| above 0.75 to 1 | 6,680 | 1,725 | 25.82 |
| above 1 | 2,115 | 636 | 30.07 |

```
cards['utilisation'] = cards['BILL_AMT1'] / cards['LIMIT_BAL']
```

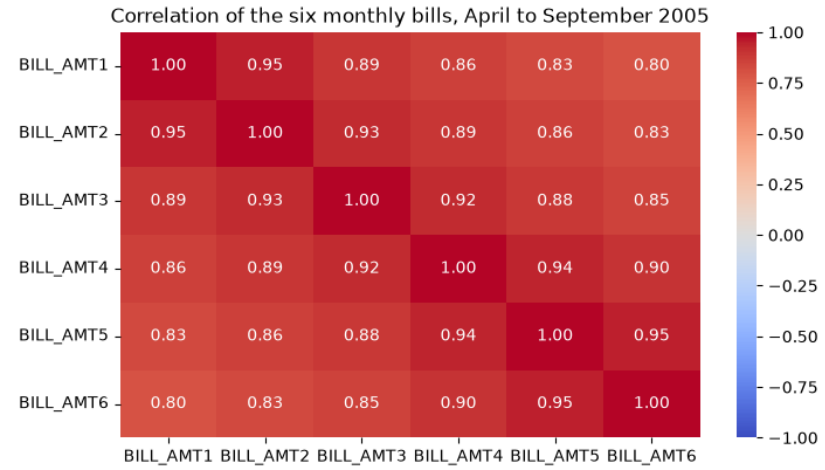
### Share of the bill paid: look at the denominator

(cards['PAY_AMT1'] / cards['BILL_AMT1']).mean() prints inf, with no error: 2,008 bills are exactly zero. On the 27,402 bills above zero the median share is 0.061 and the mean 1.776. Quote the median.

| | | | | | |
|---|---|---|---|---|---|
| `=M2 / B2` | 0.196 for the first account | `=COUNTIF(G2:L2, ">=1")` | 2 for the first account | `=MEDIAN(AK2:AK30001)` | returns 0.061 |
| `=S2 / M2` | #DIV/0! on the 2,008 rows where M is 0 | `=IF(M2 > 0, S2 / M2, "")` | blank on 2,598 rows. AK is that helper column. | | |

# Correlation: a first sort

The six monthly bills: 0.80 or more with each other




One month apart: 0.92 to 0.95. Six columns, nearly the same thing.

```
bill_corr = cards[bill_columns].corr()
with_outcome = cards[columns_to_check].corr()['default']
```

### The same in Excel

`=CORREL(M2:M30001, N2:N30001)` returns 0.95

`=CORREL(B2:B30001, Y2:Y30001)` returns -0.154

Colours: Home, Conditional Formatting, Color Scales.

Correlation with the outcome

| Column | Correlation |
|---|---|
| months_delayed | 0.398 |
| PAY_0 | 0.325 |
| utilisation | 0.086 |
| AGE | 0.014 |
| BILL_AMT1 | -0.020 |
| PAY_AMT1 | -0.073 |
| LIMIT_BAL | -0.154 |

Age: the rates behind 0.014

| age_band | accounts | rate_pct |
|---|---|---|
| 21 to 30 | 11,013 | 22.44 |
| 31 to 40 | 10,713 | 20.43 |
| 41 to 50 | 6,005 | 23.30 |
| 51 to 60 | 1,997 | 25.24 |
| 61 to 79 | 272 | 26.84 |

Partly the dip. Mostly the 0/1 outcome: limit bands from 31.79 to 13.98 percent get only -0.154.

### What a correlation with a 0/1 outcome can and cannot say

Can: give a direction, for a first sort. Not on the scale of the bills.

Cannot: say what the default rate was in any group. The table does.

Cannot: see a pattern that is not a straight line, as with age.

Treats a code as an amount: PAY_0 as evenly spaced numbers.

Says nothing about cause.

**A first sort. The rate tables, with their counts, are the findings.**

# The written observations

**Each line is a rate in this file with the number of accounts behind it. None is a statement of cause or a basis for a lending decision.**

| | |
|---|---|
| **1. The default rate** | 6,636 of the 30,000 accounts defaulted the following month: 22.12 percent. |
| **2. September status** | A delay of two months, code 2: 69.14 percent of 2,667 accounts. A delay of one month, code 1: 33.95 percent of 3,688. Pay duly, code -1: 16.78 percent of 5,686. |
| **3. Months with a delay** | No month of delay in the six: 11.71 percent of 19,931 accounts. A delay in all six: 70.32 percent of 1,341. |
| **4. Credit limit** | 50,000 NT dollars or less: 31.79 percent of 7,676 accounts. Above 240,000: 13.98 percent of 7,067. Within a September status of 2: 67.5 to 71.1 percent in all four limit bands. |
| **5. Utilisation** | Bill above zero and up to a quarter of the limit: 16.81 percent of 11,440 accounts. Bill above the limit: 30.07 percent of 2,115. |
| **6. Demographic columns** | Sex: 24.17 percent (male, 11,888 accounts), 20.78 (female, 18,112). Education: 19.23 (graduate school, 10,585), 23.73 (university, 14,030), 25.16 (high school, 4,917). Marital status: 23.47 (married, 13,659), 20.93 (single, 15,964). Age band: 20.43 (31 to 40, 10,713), 26.84 (61 to 79, 272). |
| **7. What is left open** | 17,496 accounts hold a September code, -2 or 0, that the documentation does not define: 13.23 percent (code -2, 2,759 accounts) and 12.81 percent (code 0, 14,737). Any reading of this file depends on what those codes mean. |

`cards.groupby(column)['default'].agg(['count', 'sum', 'mean'])` is behind each line. In Excel: COUNTIF, SUMIF, and AVERAGEIF on the same condition.

# The AI check

Every notebook ends with code written in the style of an AI assistant. It has one bug. Find it.

1

## Read

Read the code before you run it.

2

## Predict

Decide what it should produce.

3

## Run

Run it and look at the output.

4

## Check

Compare with a number you worked out yourself.

*Code that runs is not the same as code that is right.*

# What you can do now

- Check a file against its documentation, and keep the codes it does not define
- Turn a 0/1 column into a rate, and show its count beside it
- Find the rate for each group, and for bands made with pd.cut and pd.qcut
- Read a table of rates beside its table of counts
- Write up what a file shows, and stop where the file stops

**Week 4, Day 2** Analyzing Text, Part 1. Strings and regular expressions, on FOMC statements